

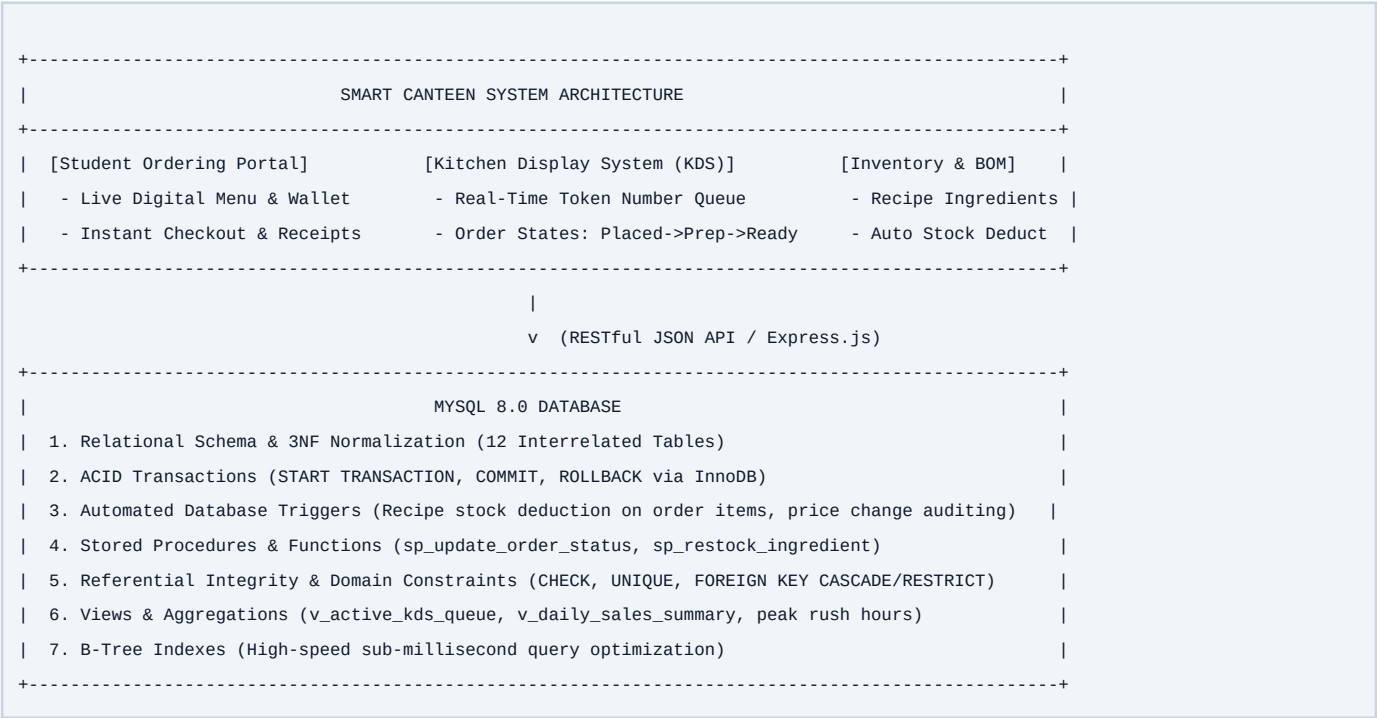
SMART CANTEEN MANAGEMENT SYSTEM

Comprehensive Database Management Systems (DBMS) Project Report & Implementation Guide

Lead Developer:	Shagun Singh (Reg No: 24BCY10379)
Target Database:	MySQL 8.0 (InnoDB Transactional Storage Engine)
Database Schema:	canteen_db (12 Normalized Relational Tables)
Full Stack Stack:	HTML5 / Modern Dark Glassmorphism CSS / Node.js Express REST API
Core DBMS Principles:	3NF Normalization, ACID Transactions, Triggers, Stored Procedures, Views, B-Tree Indexes

System Architecture & Data Flow

The Smart Canteen Management System operates on a multi-tier client-server architecture where user actions on the student ordering portal and kitchen display system (KDS) interact with a MySQL 8.0 relational database. All business logic (stock deduction, price auditing, token queues) is maintained directly through database constraints, stored procedures, and automated triggers.



PART 1: Core DBMS Concepts Implemented & Where They Work

1. Relational Schema & 3NF Normalization

In Simple Words: Splitting information into clean, specialized tables linked by IDs so data is never duplicated or inconsistent.

Where It Is Used in Code:

- **students:** Stores only student records (registration number, full name, email, wallet balance).
- **categories:** Stores food categories (South Indian, North Indian, Snacks, Beverages).
- **menu_items:** Stores dishes, prices, and preparation times.
- **ingredients:** Stores kitchen inventory raw materials (Rice Batter, Coffee Beans, Paneer, Milk).
- **item_ingredients (Bill of Materials):** Resolves the **Many-to-Many (M:N)** relationship between dishes and raw materials. One dish requires multiple ingredients; one ingredient is shared across multiple dishes.
- **orders & order_items:** Separates order headers from line items. A student can order 3 items in 1 checkout without repeating student details (1NF, 2NF, 3NF compliance).

2. Primary Keys, Foreign Keys & Referential Integrity

In Simple Words: Primary keys are unique barcodes for each row. Foreign keys link tables together and prevent invalid records.

Where It Is Used in Code:

- **students.student_id (PK) -> orders.student_id (FK):** An unregistered student can never place an order.
- **categories.category_id -> menu_items.category_id with ON DELETE CASCADE:** If a category is removed, its items are safely managed.
- **ingredients.ingredient_id -> item_ingredients.ingredient_id with ON DELETE RESTRICT:** The database strictly prevents accidental deletion of 'Milk' or 'Rice' if an existing recipe relies on it!

3. Domain Integrity & Validation Constraints

In Simple Words: Strict rules written into MySQL so negative balances or wrong numbers are rejected automatically.

Where It Is Used in Code:

- **CHECK (wallet_balance >= 0):** Guarantees a student's campus wallet balance can never fall below zero.
- **CHECK (price > 0):** Ensures all menu items have a valid positive selling price.
- **CHECK (rating BETWEEN 1 AND 5):** In the feedback table, ensures star reviews are strictly between 1 and 5.
- **UNIQUE (email) & UNIQUE (reg_no):** Rejects duplicate student or staff registrations.
- **ENUM Domains:** Restricts statuses to valid workflow stages: orders.status ('Placed', 'Preparing', 'Ready', 'Picked Up', 'Cancelled') and payments.payment_method ('Student Wallet', 'UPI', 'Card', 'Cash').

4. ACID Transactions (Transaction Processing)

In Simple Words: Ensuring multi-step purchases succeed completely or fail completely with zero lost money or half-created orders.

Where It Is Used in Code (server/index.js & database/04_transactions_demo.sql):

- **Atomicity:** Placing an order requires: (1) deducting student balance, (2) creating the order record, (3) creating order line items, and (4) deducting recipe stock. If any step fails, ROLLBACK undoes everything.
- **Consistency:** The database moves from one valid state to another, satisfying all CHECK and FK constraints.
- **Isolation:** InnoDB row-level locking ensures that if two students order the last Samosa simultaneously, race conditions cannot oversell stock.
- **Durability:** Once COMMIT is executed, the transaction is permanently written to the redo log and survives server restarts.

5. Database Triggers (Active Database)

In Simple Words: Programs stored in MySQL that run automatically whenever a table is modified.

Where It Is Used in Code (database/03_procedures_and_triggers.sql):

- **trg_check_and_deduct_stock (AFTER INSERT ON order_items):** When a dish is ordered, this trigger iterates through its recipe in item_ingredients, verifies stock availability, subtracts the required amounts from ingredients, and logs the change into inventory_logs. If stock is missing, it raises SIGNAL SQLSTATE '45000' aborting the order.
- **trg_audit_price_change (BEFORE UPDATE ON menu_items):** Every time a dish's price is updated, this trigger automatically logs the old price, new price, user, and timestamp into price_audit_log for financial transparency.

6. Stored Procedures & User-Defined Functions

In Simple Words: Pre-compiled SQL functions on the server that can be executed with a single call.

Where It Is Used in Code (database/03_procedures_and_triggers.sql):

- **sp_update_order_status(order_id, new_status):** Safely advances order status from Placed to Preparing, Ready, or Picked Up.
- **sp_restock_ingredient(ingredient_id, quantity):** Restocks raw material stock and logs the restock transaction.
- **fn_calculate_bill(subtotal, tax_rate):** Computes order totals with campus facility charges.

7. Views & Analytics Aggregations (JOINS, GROUP BY, SUM, AVG)

In Simple Words: Saved virtual tables that combine data and compute business summaries without manual calculations.

Where It Is Used in Code (database/05_analytics_views_queries.sql):

- **v_active_kds_queue:** Combines orders, students, and order_items to show token tickets on the Kitchen Display System.
- **v_daily_sales_summary:** Calculates gross revenue, total token count, and average order size.
- **v_inventory_stock_status:** Compares stock against threshold to flag 'OK', 'LOW STOCK', or 'CRITICAL'.
- **Peak Rush Hour Query:** Groups orders by HOUR(order_date) to display busiest times of day.

8. Storage Engine & Indexing

In Simple Words: B-Tree indexes speed up lookups like a book index, finding records in microseconds.

Where It Is Used in Code (database/01_schema.sql):

- **idx_orders_date_status ON orders(order_date, status):** Makes the live kitchen queue load instantly.
- **idx_orders_student ON orders(student_id):** Speeds up student token queries.
- **idx_ingredients_stock ON ingredients(current_stock, min_threshold):** Powers real-time low-stock alerts.

PART 2: Complete Database Schema Reference (12 Tables)

The system is structured across 12 normalized tables adhering to 3NF standards:

Table Name	Primary Key	Foreign Keys / Key Columns	Role in Canteen System
students	student_id	reg_no (UQ), email (UQ), wallet_balance	Student accounts, Google login, campus wallet balance
categories	category_id	category_name (UQ), description	Food menu categories (South Indian, Beverages, etc.)
menu_items	item_id	category_id (FK), item_name (UQ), price	Dishes available on the student ordering catalog
ingredients	ingredient_id	ingredient_name (UQ), current_stock, min_threshold	Raw materials inventory levels in kitchen store
item_ingredients	(item_id, ingredient_id)	item_id (FK), ingredient_id (FK), quantity_required	Bill of Materials (BOM) dish recipes for auto deduction
orders	order_id	student_id (FK), status, total_amount, order_date	Order records and official Token Numbers for KDS
order_items	order_item_id	order_id (FK), item_id (FK), quantity, subtotal	Individual dishes included within an order
payments	payment_id	order_id (FK, UQ), payment_method, transaction_ref	Payment audit receipts and payment transaction references
kitchen_staff	staff_id	email (UQ), role, is_active	Chef and kitchen staff portal login authentication
feedback	feedback_id	order_id (FK), student_id (FK), rating (1-5)	Customer ratings and food quality reviews
inventory_logs	log_id	ingredient_id (FK), change_type, quantity_changed	Audit trail of all raw material deductions & restocks
price_audit_log	audit_id	item_id, old_price, new_price, changed_at	Automatic trigger audit trail of dish price modifications

PART 3: Step-by-Step MySQL Workbench Editing Guide

You can manage your canteen database in MySQL Workbench through two simple methods: **Visual Spreadsheet Grid Editing** (no SQL knowledge required) or **Direct SQL Commands**.

Method 1: Visual Spreadsheet Editing (Like Microsoft Excel)

- 1. Connect to MySQL Server:** Open MySQL Workbench and double-click **Local instance 3306**. Enter your MySQL root password (11658369_Sql).
- 2. Locate Database Tables:** In the left sidebar under **Navigator**, click the **Schemas** tab. Expand **canteen_db** > Expand **Tables**.
- 3. Open Table for Editing:** Hover over any table (e.g., students, menu_items, ingredients) and click the **tiny Table Grid icon** on the right side. A spreadsheet will open.
- 4. How to Add a Row:** Scroll down to the bottom empty row marked with an asterisk (*). Double-click the cells and type your values. Leave auto-increment ID columns blank. Click the blue **Apply** button at the bottom-right > **Apply** > **Finish**.
- 5. How to Edit a Row:** Double-click any cell you want to change (e.g. modify a student's wallet_balance or a dish's price). Type the new value and press Enter. Click **Apply** > **Apply** > **Finish**.
- 6. How to Delete a Row:** Right-click the grey row number box on the far left of the row you wish to delete. Choose **Delete Row(s)**. Click the blue **Apply** button > **Apply** > **Finish**.

Method 2: Quick Copy-Paste SQL Commands for Each Table

Open a new SQL query tab in MySQL Workbench (**Ctrl + T**), write USE canteen_db;, and run any command below:

1. Managing Students (students table)

```
-- View all registered students
SELECT student_id, reg_no, full_name, email, wallet_balance FROM canteen_db.students;

-- Add a new student
INSERT INTO canteen_db.students (reg_no, full_name, email, phone, wallet_balance)
VALUES ('24BCY99999', 'Priya Patel', 'priya.patel@campus.edu', '9876543210', 500.00);

-- Add money to a student's wallet (e.g. Add Rs. 500 to Shagun Singh)
UPDATE canteen_db.students
SET wallet_balance = wallet_balance + 500.00
WHERE email = 'shagun.24bcy10379@vitbhopal.ac.in';

-- Delete a student
DELETE FROM canteen_db.students WHERE reg_no = '24BCY99999';
```

2. Managing Kitchen Staff (kitchen_staff table)

```
-- View all staff members
SELECT * FROM canteen_db.kitchen_staff;

-- Add a new chef
INSERT INTO canteen_db.kitchen_staff (full_name, email, role, is_active)
VALUES ('Vikram Singh', 'vikram.chef@canteen.campus.edu', 'Cook', 1);

-- Promote staff to Canteen Manager
UPDATE canteen_db.kitchen_staff
SET role = 'Canteen Manager'
WHERE email = 'shagun.24bcy10379@vitbhopal.ac.in';

-- Remove a staff member
DELETE FROM canteen_db.kitchen_staff WHERE email = 'vikram.chef@canteen.campus.edu';
```

3. Managing Canteen Menu Dishes (menu_items table)

```
-- View all menu items
SELECT item_id, item_name, price, is_available FROM canteen_db.menu_items;

-- Add a new dish (1=South Indian, 2=North Indian, 3=Snacks, 4=Beverages)
INSERT INTO canteen_db.menu_items (category_id, item_name, description, price, preparation_time_mins, is_available)
VALUES (3, 'Veg Grilled Sandwich', 'Toasted triple-layer sandwich with fresh veggies', 60.00, 8, 1);

-- Update dish price (Automatically logged in price_audit_log via Trigger!)
UPDATE canteen_db.menu_items
SET price = 70.00
WHERE item_name = 'Masala Dosa';

-- Mark a dish as Sold Out (0) or Available (1)
UPDATE canteen_db.menu_items
SET is_available = 0
WHERE item_name = 'Mango Lassi';

-- Delete a dish
DELETE FROM canteen_db.menu_items WHERE item_name = 'Veg Grilled Sandwich';
```

4. Managing Raw Kitchen Stock (ingredients table)

```
-- View inventory stock levels and alert thresholds
SELECT ingredient_id, ingredient_name, current_stock, unit, min_threshold
FROM canteen_db.ingredients;

-- Restock raw materials (e.g. Add 5,000 grams of Paneer)
UPDATE canteen_db.ingredients
SET current_stock = current_stock + 5000.00
WHERE ingredient_name = 'Paneer';

-- Add a new raw grocery ingredient
INSERT INTO canteen_db.ingredients (ingredient_name, unit, current_stock, min_threshold, cost_per_unit)
VALUES ('Butter', 'grams', 3000.00, 500.00, 0.60);
```

5. Tracking Orders & Kitchen Tokens (orders table)

```
-- View active kitchen queue orders sorted by Token Number
SELECT order_id AS token_no, status, total_amount, order_date
FROM canteen_db.orders
WHERE status != 'Picked Up'
ORDER BY order_id ASC;

-- Manually advance an order status to Ready for Pickup
UPDATE canteen_db.orders
SET status = 'Ready'
WHERE order_id = 1;
```

Golden Rules to Remember:

- **1. Always Click the Blue 'Apply' Button:** In MySQL Workbench's visual editor, edits are not committed to disk until you click Apply.
- **2. Leave Auto-Increment IDs Blank:** Columns like student_id, item_id, and order_id are numbered automatically by MySQL.
- **3. Foreign Key Protection:** You cannot delete an ingredient if an existing dish recipe depends on it; delete the recipe row in item_ingredients first.
- **4. Live Instant Synchronization:** Any update you make in MySQL Workbench reflects in the web application immediately upon refreshing the page!